
Busy Beaver: Ai limiti del computabile

Saggio sui limiti logici della matematica

Autore:

Benedetto Miazzo

Data:

10 Maggio 2026

BUSY BEAVER

Per affrontare questo argomento è necessario introdurre in concetto di Macchina di Turing

Macchina di turing

E' un modello matematico di computazione ideato dal Matematico Alan Turing al fine di Formalizzare il concetto di algoritmo, descrivendo in modo preciso cosa fa un "calcolatore" (anche umano) quando segue istruzioni passo passo. Capire i limiti del calcolo: dimostrò che esistono problemi che nessuna macchina può risolvere. Tutto cio' nel contesto di rispondere al problema della decisione: contribuì a dimostrare che non esiste un metodo universale per risolvere tutti i problemi matematici.

Funzionamento:

La macchina di turing modella matematicamente una macchina meccanica la quale è a sua volta una idealizzazione di una cpu. La macchina opera su un nastro infinito sul quale sono presenti simboli che la macchina può leggere e scrivere, uno alla volta. Le operazioni della macchina sono determinate da un insieme finito di istruzioni:

Un esempio di istruzione potrebbe essere: Stato 10, se il simbolo letto è 1, scrivi 1, se il simbolo letto è 0, passa allo stato 17

Nel dettaglio;

Nastro

Il nastro è diviso in celle, una affiancata all'altra. Ogni cella contiene un simbolo facente parte di un dato alfabeto finito, questo alfabeto contiene inoltre uno speciale simbolo "vuoto" a cui mi riferirò come 0 ed uno o più altri simboli. Il nastro è considerato infinito in quanto sempre estendibile sia a destra che a sinistra per fornire alla macchina nastro. Celle in cui non si è mai scritto si assume che contengano il simbolo vuoto

Testa

la quale può leggere e scrivere i simboli e muoversi a destra o sinistra di una ed una sola cella soltanto per volta

Registro dello Stato

Contiene lo stato attuale della macchina, tra potenzialmente infiniti, tra questi stati è presente lo stato di partenza a cui è inizializzato il registro e lo stato di terminazione (HALT) della macchina.

Tabella degli Stati

contiene un numero finito di istruzioni, le quali dato lo stato attuale della macchina e il simbolo che la macchina sta leggendo in questo momento dicono alla macchina di fare, in ordine, le seguenti azioni:

- Cancellare o scrivere un simbolo,
- Muovere la testa a destra (R), sinistra (L) oppure rimanere ferma (N),
- Assumere lo stesso o un nuovo stato.

DEFINIZIONE FORMALE

Una macchina di Turing (a nastro singolo) può essere definita formalmente come una 7-upla $M = \langle Q, \Gamma, b, \Sigma, \delta, q_0, F \rangle$ dove:

- Q è un insieme finito e non vuoto di **stati**;
- Γ è un insieme finito e non vuoto di **simboli dell'alfabeto del nastro**;
- $b \in \Gamma$ è il **simbolo vuoto** (l'unico simbolo a cui è permesso apparire sul nastro infinite volte in qualsiasi fase della computazione);
- $\Sigma \subseteq \Gamma \setminus \{b\}$ è l'insieme dei **simboli di input**, ovvero l'insieme dei simboli a cui è permesso apparire nel contenuto iniziale del nastro;
- $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ è una **funzione parziale** chiamata funzione di transizione, dove L è lo spostamento a sinistra e R lo spostamento a destra. Se δ non è definita sullo stato corrente e sul simbolo corrente del nastro, la macchina si ferma; intuitivamente, la funzione di transizione specifica lo stato successivo, quale simbolo sovrascrivere al simbolo corrente puntato dalla testa e il movimento successivo della testa;
- $q_0 \in Q$ è lo **stato iniziale**;

- $F \subseteq Q$ è l'insieme degli **stati finali** o **stati di accettazione**. Il contenuto iniziale del nastro si dice accettato da M se la macchina alla fine si ferma in uno stato appartenente a F .

PROPIETA'

Tesi di Church-Turing

Questa tesi (che non è un teorema dimostrabile, ma un assioma universalmente accettato dalla comunità scientifica) afferma che: Qualunque algoritmo o procedura effettiva di calcolo è formalizzabile mediante una Macchina di Turing.

Equivalenza dei Modelli di Calcolo

Tutti i modelli di calcolo possono essere ricondotti ad una macchina di turing, pertanto hanno lo stesso potere computazionale

nel contesto della gerarchia di chomsky la macchina di turing ricade nel livello 0

BUSY BEAVER

Il gioco a n -stati del busy beaver ($BB - n$ game) fu' introdotto da Tibor Rado' nel 1962 e riguarda una classe di macchine di turing composta dalle macchine con:

- n -stati
- un alfabeto composto da $\{0, 1\}$ con 0 come simbolo vuoto (blank)

“Eseguire” la macchina consiste nell’ partire dallo stato iniziale su una cella qualsiasi del nastro, contenente solo celle vuote (0), e poi iterare la funzione di transizione finche’ lo stato di HALT e’ raggiunto. Se e solo se una macchina termina di eseguire, il numero di 1 rimasti sul nastro verra’ chiamato il punteggio della macchina. l’ N esimo busy beaver, $BB - n$ e’ una macchina che vince il gioco del n -state busy beaver game. A seconda della definizione la macchina vince, ottenendo il massimo numero di segni sul nastro denominato $\Sigma(n)$ oppure eseguendo più a lungo $S(n)$, di tutte le altre possibili macchine ad n -stati. La funzione $BB - n$ e’ la funzione che cresce più rapidamente che esista ed e funge da limite per ciò che e’ computabile e cosa no.

APPLICAZIONI

Il busy beaver offre un approccio completamente nuovo per affrontare problemi di matematica pura. Di fatto molti problemi aperti potrebbero in teoria, ma non in pratica, essere risolti sistematicamente conoscendo il valore di $S(n)$ per n sufficientemente grande. In altre parole $S(n)$ codifica la risposta a tutti le congetture matematiche che possono essere verificate in un tempo infinito da una macchina di turing con un numero di stati minore o uguale a n .

Prendiamo ad esempio una qualsiasi congettura Π_{10} , che possa essere provata falsa tramite controesempio in un numero finito di casi. Scriviamo un programma che sequenzialmente verifica questa congettura per valori sempre maggiori, terminando qualora incontri un controesempio, se la congettura invece e' vera il programma continuerà all'infinito.

Grazie alla tesi di Church-Turing possiamo simulare questo programma con una macchina di turing a n -stati. Conoscendo $S(n)$ possiamo decidere in un tempo finito di tempo se la macchina si fermerà (la congettura e' falsa) oppure se non si fermerà mai (la congettura e' vera).

Si può codificare la verifica della congettura di Goldbach in una macchina di Turing a 25 stati. Il processo di calcolo è strutturato per terminare (*halt*) se e solo se la congettura è falsa. Conoscendo $S(25)$ potremmo verificare in una quantità finita di tempo se la congettura e' vera o meno.

Ottenere il valore di $S(25)$ e' pero' tutt'altro che facile.

E' stato provato che non esiste modo di trovare algoritmicamente il valore di $S(n)$, non esiste una macchina di turing che preso in ingresso un numero, restituisca $S(n)$. Tutti i valori conosciuti di $S(n)$ sono stati provati enumerando ogni macchina ad n -stati e provando che termina o meno.

Sarebbe necessario calcolare $S(n)$ attraverso un metodo meno diretto perché sia utile. Inoltre i valori di $S(n)$ e di $\Sigma(n)$ diventano estremamente grandi estremamente velocemente. $S(5)$ equivale a circa 47 milioni

$S(6)$ il quale e' almeno maggiore di $2 \uparrow \uparrow \uparrow 5$, il quale e' un numero incomprensibilmente grande.

Limiti della computazione

Le proprietà di crescita della funzione Busy Beaver hanno implicazioni sul comportamento dei sistemi fisici. Assumendo che la tesi di Church-Turing sia vera e tutte le funzioni fisica-

mente computabili sono Turing-compatibili, allora nessuna quantità fisica, direttamente misurabile, può crescere più velocemente della funzione Busy Beaver.

Sviluppi Recenti

E' stato provato che $S(5)$ equivale a 47.176.870, questo risultato e' stato possibile grazie alla <https://bbchallenge.org/> la quale ha applicato la struttura collaborativa di un progetto informatico open-source alla risoluzione di questo complesso problema matematico. La prova formale e' stata codificata in coq.

Le tecniche utilizzate per risolvere $BB(5)$ illustrano la facilità con cui il gioco Busy Beaver genera piccoli problemi matematici aperti e non banali, che mettono alla prova la nostra intelligenza e i limiti della conoscenza matematica. I problemi aperti generati dal lavoro della bbchallenge sono stati inclusi in un dataset di congetture formalizzate in Lean, progettati per servire come sfide future per gli strumenti di ragionamento dell'IA [22]. Più in generale, data l'enorme quantità di problemi, con un'ampia varietà di difficoltà (da molto facili a quasi impossibili), che il gioco Busy Beaver può generare, la capacità di dimostrare valori noti di Busy Beaver o di fare progressi su quelli sconosciuti rappresenterebbe un obiettivo ambizioso per l'intelligenza artificiale. Coq- $BB(5)$ ha già iniziato ad essere utilizzato con questo obiettivo in mente: i sistemi di IA sono stati testati sulla dimostrazione dei primi 100 lemmi della dimostrazione $S(4)$, con un successo del 58% finora.

CONCLUSIONI, Indipendenza Logica e Limiti di ZFC

Il valore di $BB(n)$ non è solo difficile da calcolare, ma oltre una certa soglia diventa matematicamente dimostrabile. È stato dimostrato (Yedidia e Aaronson, 2016) che esiste un valore k per cui il valore di $BB(n)$ è indipendente dagli assiomi ZFC (la teoria degli insiemi di Zermelo-Fraenkel con l'assioma della scelta). Questo significa che, all'interno del sistema logico standard usato dalla matematica moderna, non è possibile né dimostrare che $BB(n)$ valga k , né che sia diverso da k .

Evoluzione del limite: La stima iniziale per questa soglia era di 7910 stati. Ricerche successive (Stefan O'Rear) hanno ridotto drasticamente questo numero a meno di 1000 stati.

Implicazioni: Una macchina di Turing con meno di 1000 stati può essere progettata per cercare una contraddizione negli assiomi della matematica stessa. Se la matematica è coerente, la macchina non si fermerà mai, ma ZFC non può dimostrare la propria coerenza (per il Secondo Teorema di Incompletezza di Gödel). Di conseguenza, il comportamento di

quella specifica macchina — e quindi il valore di $BB(n)$ — rimarrà per sempre sospeso nell'indcidibilità logica. In sintesi, il Busy Beaver non rappresenta soltanto una sfida per la potenza dei nostri computer, ma evidenzia il punto di rottura oltre il quale le nostre leggi logiche non possono più fornire risposte certe. Oltre una soglia minima di stati, la macchina di Turing smette di essere un semplice esecutore di istruzioni prevedibili per diventare un sistema il cui comportamento è intrinsecamente legato alla struttura profonda della matematica. In questo contesto, il valore di $BB(n)$ traccia il confine dove la verità e la dimostrabilità si separano. Se una macchina di Turing è progettata per fermarsi solo se gli assiomi della nostra logica fossero inconsistenti, la nostra incapacità di determinare se tale macchina termini o meno non è un limite tecnologico, ma un limite logico fondamentale. Il Busy Beaver agisce quindi come una misura oggettiva della nostra ignoranza: ogni stato aggiuntivo non aumenta solo la difficoltà del calcolo, ma ci spinge in un dominio dove le domande che poniamo eccedono la capacità di risposta del sistema formale che abbiamo costruito per interpretarle.

Fonti:

- <https://arxiv.org/pdf/2509.12337>
- https://en.wikipedia.org/wiki/Busy_beaver